The Importance of Monitoring

1. Overview

Monitoring is one of the **most critical pillars** of production systems.

A system without monitoring is like:

> Flying an aircraft with no dashboard.

Even if architecture is excellent, without visibility you won't know:

- Failures
- Slowdowns
- Resource exhaustion
- Security anomalies
- Customer pain

2. Definition

Monitoring is the continuous observation of system health, performance, reliability, and user experience using metrics, logs, traces, and alerts.

3. Why Monitoring Matters

Monitoring helps detect:

Technical Problems

- Server crashes

- High CPU / memory
- Database slow queries
- Queue backlog
- Cache failures

Performance Problems

- Increased latency
- Timeout spikes
- Throughput drops

User Experience Problems

- Checkout failures
- Login issues
- Video buffering
- Push notifications delayed

Security Problems

- Suspicious traffic
- Brute-force login attempts
- Data access anomalies

4. Golden Rule

> If you cannot measure it, you cannot improve it.

5. Customer Experience is the Real Metric

System may look healthy internally, but users may suffer.

Examples:

- APIs return 200 but wrong data
- UI loads slowly due to CDN issue
- Payment success internally but user sees failure screen

So monitoring should include:

External Signals

- Synthetic checks
- Real User Monitoring (RUM)
- Conversion funnels

6. Core Monitoring Pillars

A. Metrics

Numeric time-series data.

Examples:

- Requests/sec
- Error %
- CPU usage
- Memory usage
- DB connections
- Cache hit rate

B. Logs

Detailed event records.

Examples:

```
None
User 123 login failed

Order 456 payment timeout
```

C. Traces

Track request across microservices.

```
None
Client → API Gateway → Auth → Order → Payment
```

Useful for latency bottlenecks.

D. Alerts

Automatic notifications when thresholds breached.

Examples:

- Error rate > 5%
- P99 latency > 800ms
- Disk > 90%

7. Incident Priority Levels

You mentioned urgency - very important.

P0 (Critical)

- Entire service down
- Payment broken
- Massive outage

Immediate action.

P1 (High)

- Major degradation
- Region down
- Slow checkout

P2 (Medium)

- Non-critical feature broken

P3 (Low)

- Cosmetic issue
- Minor metric anomaly

8. Monitoring Dependencies

If your service is used by others:

```
Payments Service ← Order Service ← Checkout
```

When checkout breaks, they may blame payments.

You need monitoring to quickly prove:

- Was your service healthy?
- Was latency elevated?
- Any errors?

9. What to Monitor

Application Layer

- QPS
- Success/error rate
- Latency P50/P95/P99

Infrastructure Layer

- CPU
- Memory
- Disk
- Network

Database Layer

- Query latency
- Locks
- Replication lag
- Connections

Cache Layer

- Hit ratio
- Evictions
- Memory usage

Queue Layer

- Consumer lag
- Retry count
- DLQ growth

Business Layer

- Orders/min
- Revenue/hour
- Signup success %
- Churn rate

10. SLI / SLA / SLO

SLI

Measured metric.

Example:

- 99.9% successful requests

SLO

Target objective.

Example:

- P99 latency < 300ms

SLA

Contract with customer.

Example:

- 99.95% uptime

11. Example Dashboard

```
Traffic: 15k RPS

Error Rate: 0.3%

P99 Latency: 220ms

CPU: 62%

DB Lag: 0 sec
```

12. Good Monitoring Stack Examples

- Prometheus + Grafana
- Datadog
- New Relic
- ELK Stack
- OpenTelemetry

13. Interview Script

Say:

> “I’d add observability from day one: metrics, logs, traces, dashboards, and alerting. I’d monitor not only infrastructure but also user-facing KPIs like checkout success and P99 latency.”

Strong answer.

14. Final Summary

- Monitoring provides production visibility
- Detects failures before users complain
- Includes metrics, logs, traces, alerts
- Monitor technical + business KPIs
- Critical for debugging dependencies

15. Memorable Line

> Hope is not a monitoring strategy.